CSA15 - CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 1 - CO1

Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System

Student Name	SHANE MAC S	Register Number	192211511
Department / Section	CSE	Academic Year / Semester	2O22
Faculty Name	ANTONY JOSEPH RAJAN D	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date		Evaluation Date	

CO1: Apply cloud computing technologies including hardware-software evolution, virtualization, web services, and service models to develop cloud-based solutions.

Assumptions used throughout this report

The scenario gives some figures; the rest are stated assumptions so that every calculation can be checked. All cost figures are illustrative planning estimates in Indian Rupees (INR) lakh (1 lakh = 100,000) and should be re-checked with the provider's pricing calculator before real budgeting.

- 12 existing physical servers, one per application component, 15% average CPU utilisation, peaks above 95% during admissions, result publication and exam weeks.
- About 27,000 users (25,000 students + 2,000 staff) on 4 campuses. Current peak is assumed to be about 200 HTTP requests per second across all applications.
- Day-4 scenario: 40 affiliated colleges and a mobile app add 10 times the current peak load (about 2,000 requests per second) and require 99.9% availability.
- A cloud region in India is used so that student and fee data stay within the country.

TASK 1 - Introduce Cloud Computing and Map It to the Scenario (3 Marks)

1.1 Definition

Cloud computing, as defined by NIST (SP 800-145), is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (networks, servers, storage, applications and services) that can be rapidly provisioned and released with minimal management effort or service-provider interaction. In simple terms, the university rents computing power over the internet and pays only for what it uses, instead of buying and running its own servers.

1.2 The five essential characteristics

Table 1: Five essential characteristics and their meaning for the university

Characteristic	Meaning	University example
On-demand self-service	Users provision resources themselves, without human interaction with the provider.	IT staff launch a new server in minutes instead of waiting 6-8 weeks for procurement.
Broad network access	Services are reachable over the network from many device types.	Students use the mobile app, laptops and partner colleges use APIs.
Resource pooling	Provider's resources serve many customers (multi-tenant) and are assigned dynamically.	Idle capacity is not locked to one application; the pool serves SIS, LMS and Exam Portal as needed.
Rapid elasticity	Capacity scales out and in quickly, often automatically.	Extra servers are added during admissions and exam weeks and removed afterwards.
Measured service	Usage is metered; billing and monitoring are per use.	The university pays only for compute hours actually used; cost is visible per application.

1.3 Deployment models and the model that suits the university

Table 2: Cloud deployment models compared

Model	Description	Fit for the university
Public	Owned and operated by a provider, shared by many customers over the internet.	Best for elasticity and low cost; suitable for most workloads.
Private	Dedicated to one organisation, on-premise or hosted.	High control but needs capital spend and still cannot burst; poor fit for peaks and a limited budget.
Hybrid	Combination of private and public clouds connected together.	Useful during migration and for keeping a few legacy or LAN-bound systems on campus.
Community	Shared by organisations with common concerns (e.g. education consortium).	Possible later for sharing services with affiliated colleges, but not needed now.

Recommendation: public cloud as the target model, using a hybrid arrangement during the migration. The university's problems are cost and peak capacity, which public cloud solves best. A secure VPN link to the campus network keeps any not-yet-migrated system working during the transition. Student data is kept in an India-based region and encrypted, which addresses privacy concerns.

1.4 Three problems of the on-premise setup and how cloud characteristics address them

Table 3: Mapping problems to cloud characteristics

Problem in current setup	Cloud characteristic	How it helps
1. Servers idle at 15% average utilisation and need high capital spending.	Resource pooling + measured service	Workloads share a pool, so no capacity is bought for rare peaks, and the university pays per use instead of owning idle hardware.
2. Servers fail during admissions, results and exam weeks (peaks above 95%).	Rapid elasticity	Auto-scaling adds capacity automatically at peak and releases it afterwards.
3. Slow procurement (6-8 weeks per server).	On-demand self-service	New resources are created in minutes through a console or script.
4. No public interfaces for mobile apps or partner colleges, and no DR site.	Broad network access (+ pooling across regions)	Services are exposed securely over the network, and a second region can serve as a disaster-recovery site.

1.5 Benefits and limitations of cloud adoption

Benefits:

- Lower capital expenditure (CapEx becomes pay-per-use operating expenditure, OpEx).
- Elastic capacity for peaks and faster delivery of new services.
- Built-in disaster recovery, backups and multi-zone availability that the university cannot afford on its own.
- Less routine maintenance (patching, hardware replacement), so staff can focus on services.

Limitations:

- Vendor lock-in and dependence on the provider's availability and pricing.
- Data privacy, compliance and data-location concerns for student records and fee data.
- Requires reliable internet connectivity and new skills (cloud operations, cost management).
- Unmanaged usage can cause surprise bills.

TASK 2 - Trace the Evolution and Hardware Evolution (3 Marks)

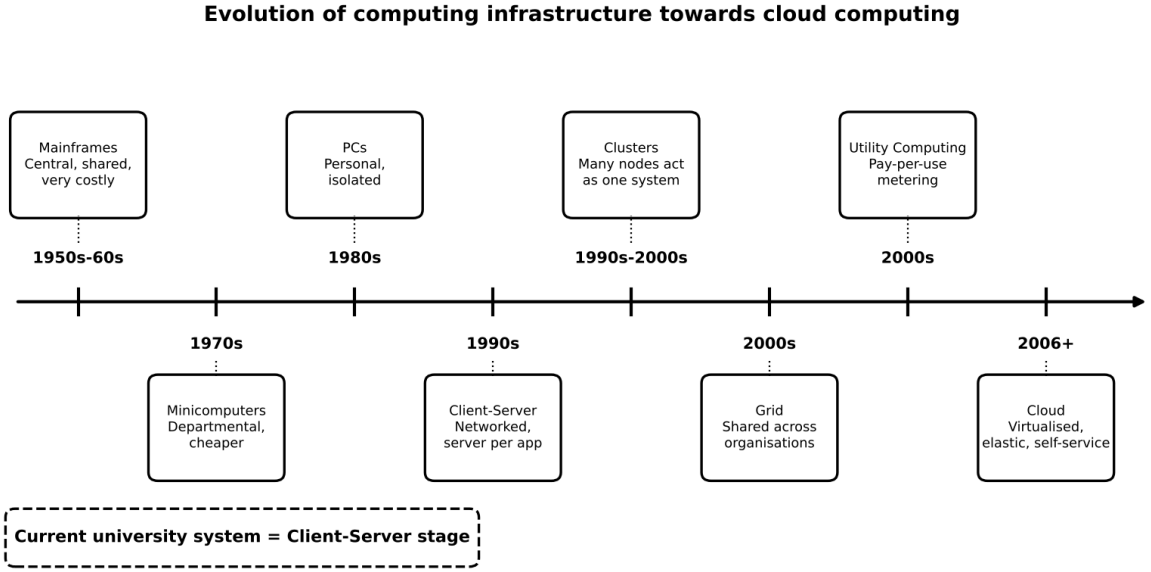


Figure 1: Timeline of computing infrastructure evolution from mainframes to cloud

2.1 How each stage solved a limitation of the previous one

Table 4: Evolution stages, limitations solved and limitations that remained

Stage	Limitation of the previous stage it solved	Limitation that remained
Mainframes (1950s-60s)	Calculations done manually or on single-purpose machines.	Huge cost, central control, only large organisations could afford them.
Minicomputers (1970s)	Mainframes were too expensive for departments.	Still costly; each department had its own isolated machine.
PCs (1980s)	Users had no personal computing power.	Isolated data; hard to share and manage.
Client-server (1990s)	PCs could not share data and applications.	One server per application, low utilisation, scaling means buying hardware.
Clusters (1990s-2000s)	A single server could not provide enough power or reliability.	Tightly coupled and owned by one organisation.
Grid computing (2000s)	Individual clusters could not be shared across organisations.	Complex to use; scheduling and billing were difficult.
Utility computing (2000s)	Grids had no simple pay-per-use model.	Little automation or self-service.
Cloud (2006 onwards)	Added virtualization, self-service, elasticity and metering.	Vendor lock-in, privacy and connectivity concerns.

2.2 Hardware evolution that enabled the cloud

Table 5: Hardware advances and their contribution to cloud computing

Hardware advance	Contribution to cloud
Multi-core processors	Many cores per CPU allow many virtual machines on one host (hardware virtualization support such as Intel VT-x and AMD-V).
Commodity servers	Cheap standard x86 servers in large numbers replaced costly specialised machines, making huge data centres affordable.
High-capacity storage (HDD to SSD, SAN, object storage)	Cheap, large and fast storage supports pay-per-GB storage and backups.
High-speed networking (fibre, 10/40/100 GbE)	Fast links let users reach distant data centres and let servers inside a data centre act as one pool.
Modern data centres	Large-scale power, cooling and automation give low cost per unit of compute and multi-zone, multi-region designs.

2.3 Which stage the university's current system corresponds to

The university's system corresponds to the **client-server stage**. The reasons are: (1) each application (SIS, LMS, Library, Exam Portal, Email) has its own dedicated physical server, which is the classic one-server-per-application pattern; (2) capacity is fixed and bought in advance, so utilisation is only about 15% on average but fails at peaks; (3) applications are large monolithic programs without public interfaces; (4) scaling requires buying hardware with a 6-8 week delay; and (5) there is no resource pooling, no metering and no disaster-recovery site. Moving to the cloud model removes each of these limits.

TASK 3 - Analyse Internet Software Evolution (3 Marks)

3.1 Evolution of internet software

Table 6: Stages of internet software evolution

Stage	Characteristics	Relevance to cloud
Static web pages (Web 1.0)	Fixed HTML pages, read-only, served from a web server.	Established HTTP/HTML and browser-based access.
Dynamic web	Server-side programs (CGI, PHP, Java servlets) and databases generate pages per user.	Web applications such as online results and registration became possible.
Web 2.0	Interactive, user-generated content, AJAX-driven rich pages, social platforms.	Applications behave like desktop software inside the browser.
SOA and web services	Functions exposed as reusable services using SOAP/REST over HTTP.	Applications become loosely coupled building blocks that can be called by others.
Mashups	New applications built by combining several services/APIs.	Shows the value of public APIs (for example maps combined with campus data).

Stage	Characteristics	Relevance to cloud
Cloud-hosted applications	Applications delivered as services (SaaS) on elastic infrastructure.	Software is rented and accessed anywhere, scaled by the provider.

3.2 Role of browsers, HTTP, AJAX and APIs

- **Browsers:** a universal client; nothing has to be installed, so any student device can use the service.
- **HTTP/HTTPS:** the standard request-response protocol that carries web pages and API calls; HTTPS adds encryption.
- **AJAX:** lets a page fetch data in the background without reloading, giving fast, desktop-like interfaces (for example checking an exam result without refreshing the page).
- **APIs:** well-defined interfaces (usually REST/JSON) that let mobile apps, partner colleges and other systems use university functions programmatically.

3.3 Desktop, client-server and web-delivered software

Table 7: Comparison of software delivery models

Aspect	Desktop software	Client-server software	Web-delivered software
Installation	On each computer	Client on each PC, server in the data centre	None; browser or thin app
Updates	Manual on every device	Client and server both updated	Central, instant for all users
Data location	Local machine	Central server	Central, in the cloud
Scalability	Per device	Limited by the fixed server	Elastic and shared
Access	Only on that device	Mostly inside campus network	Anywhere with internet
Integration	Difficult	Custom, tightly coupled	API-based, easy to integrate

3.4 Effect on the design of the university's applications

- **Break up the monoliths:** split SIS, Exam Portal and others into modules/services (results, fees, registration) that can scale independently.
- **API-first design:** each function is exposed as a REST API so the mobile app and partner colleges can use it.
- **Stateless web tier:** session data is stored in a shared cache or database so any server can answer any request, enabling auto-scaling.
- **Browser-based interfaces:** remove client software installation and work on any device.
- **Buy rather than build** commodity applications (email, LMS, library) as SaaS.

TASK 4 - Apply Server Virtualization (5 Marks)

4.1 How a hypervisor works

A hypervisor (virtual machine monitor) is a thin software layer that abstracts the physical CPU, memory, storage and network and divides them among several isolated virtual machines (VMs). Each VM has its own virtual hardware and guest operating system and believes it has a full computer. The hypervisor schedules virtual CPUs on physical cores, maps virtual memory to physical memory, and enforces isolation between VMs. This allows many under-used servers to be consolidated onto few physical hosts.

Table 8: Type-1 and Type-2 hypervisors

Feature	Type-1 (bare metal)	Type-2 (hosted)
Runs on	Directly on the hardware	On top of a host operating system
Performance	High, minimal overhead	Lower, extra OS layer
Typical use	Data centres and production clouds	Desktops, testing, learning
Examples	VMware ESXi, Microsoft Hyper-V, KVM, Xen	Oracle VirtualBox, VMware Workstation

4.2 Types of virtualization compared

Table 9: Full, para-, OS-level and hardware-assisted virtualization

Type	How it works	Advantages	Disadvantages
Full virtualization	Hypervisor fully emulates hardware; guest OS runs unmodified.	Any guest OS; strong isolation.	Some overhead from trapping privileged instructions.
Para-virtualization	Guest OS is modified to make hypercalls to the hypervisor.	Better performance than software emulation.	Needs a modified guest OS; limited OS support.
OS-level (containers)	Containers share the host kernel; isolation by namespaces and cgroups.	Very light, start in seconds, high density.	Same kernel for all; weaker isolation than VMs.
Hardware-assisted	CPU extensions (Intel VT-x, AMD-V) support virtualization directly.	Near-native speed with unmodified guests.	Needs supporting hardware (now standard).

4.3 Which applications run as VMs and which as containers

The assumed breakdown of the 12 current servers and their target form is shown below.

Table 10: Placement of the 12 servers as VMs, containers or services

Current server (assumed)	Target form	Reason
1. SIS application	VM first (rehost), later container	Legacy monolith with OS dependencies; needs minimal change at first.

Current server (assumed)	Target form	Reason
2. SIS database	VM, later DBaaS	Stateful and critical; needs strong isolation and snapshots.
3. LMS application	SaaS (or container if self-hosted)	Commodity application; vendor handles scaling.
4. LMS database	Part of SaaS (or DBaaS)	Managed by the vendor.
5. Library application + database	SaaS (or one small container)	Low load, little customisation.
6. Exam Portal web/app	Containers	Stateless, spiky load, needs fast scale-out.
7. Exam Portal database	VM, later DBaaS	Stateful and critical during exams.
8. Email server	SaaS	Commodity; not worth running ourselves.
9. File server	VM, later object storage	Needs file-system semantics at first.
10. Directory / authentication	VM (HA pair)	Core service; needs strong isolation.
11. Backup server	Replaced by cloud backup service	Provider service is cheaper and off-site.
12. Monitoring and miscellaneous tools	Containers	Light, easy to redeploy.

In short: **VMs** for stateful, legacy or security-sensitive components (databases, directory, rehosted monolith); **containers** for stateless web, API and utility components that must scale quickly; **SaaS** where the university should not run the software at all.

4.4 Consolidation calculation

Assumptions: (a) the replacement hosts have the same compute capacity as one old server (this is conservative, since modern servers are several times faster); (b) hypervisor overhead is 10%; (c) target host utilisation is 70%; (d) one extra host is added for fault tolerance (N+1); (e) during peak weeks only the SIS, Exam Portal and LMS components (5 of the 12 servers) run at 95% while the others stay at 15%.

Table 11: Worked consolidation calculation

Step	Working	Result
1. Average load of all servers	12 servers x 15%	1.8 server-equivalents (SE)
2. Add hypervisor overhead (10%)	1.8 x 1.10	1.98 SE
3. Hosts needed at 70% target utilisation	1.98 / 0.70 = 2.83	3 hosts (rounded up)
4. Add one spare host (N+1)	3 + 1	4 hosts for normal operation
5. Peak load (5 servers at 95%, 7 at 15%)	(5 x 0.95) + (7 x 0.15) = 4.75 + 1.05	5.80 SE
6. Peak with overhead	5.80 x 1.10	6.38 SE

Step	Working	Result
7. Hosts needed at peak at 70% utilisation	6.38 / 0.70 = 9.11	10 hosts (rounded up)
8. Memory sanity check (assumed 32 GB used per old server)	12 x 32 GB = 384 GB; with 20% headroom = 480 GB; 3 hosts x 256 GB = 768 GB	Memory is sufficient

Result and interpretation: the 12 servers can be consolidated onto **3 hosts (4 with a spare)** for normal operation, a reduction of about 67-75%. However, handling the peak weeks needs about **10 hosts**. Owning 10 hosts for a few weeks a year would leave 6 idle for the rest of the year. This is why elastic public-cloud capacity (renting extra hosts only during peaks) is better than buying more physical servers. The calculation is an estimate and should be validated by load-testing before final sizing.

4.5 Benefits and risks of virtualization

Table 12: Benefits and risks of virtualization

Item	Explanation
Live migration (benefit)	A running VM is moved to another host without downtime (for example VMware vMotion, Hyper-V Live Migration), allowing hardware maintenance without stopping services.
Snapshots (benefit)	The state of a VM can be saved and restored quickly, which is useful before upgrades and for fast rollback or backup.
Over-commitment (benefit)	Allocating more virtual CPUs/memory than physical capacity works because VMs rarely peak together; this raises utilisation, but must be limited (for example CPU 3:1, memory about 1.2:1) to avoid slowdowns.
Noisy neighbour (risk)	One VM using too much CPU, disk or network slows others. Mitigated by resource limits, reservations and monitoring.
Hypervisor failure (risk)	One host failure takes down all its VMs. Mitigated by clusters, high availability restart, N+1 hosts and multi-zone deployment.
Security isolation (risk)	A hypervisor vulnerability could allow VM escape. Mitigated by patching, network segmentation, sensitive workloads on dedicated hosts and least privilege.

TASK 5 - Apply Web Services (4 Marks)

5.1 Overview of web services

A web service is a software function that can be called over a network using standard web protocols (HTTP) and data formats (XML or JSON), regardless of the programming language or platform of the caller. In Service-Oriented Architecture (SOA), an application is built from loosely coupled services with published interfaces. In the cloud, web services are the way the university's applications talk to each other and to mobile apps and partner colleges, and cloud platforms themselves are controlled through APIs.

5.2 SOAP, WSDL, UDDI and REST compared

Table 13: SOAP, WSDL, UDDI and RESTful services

Technology	What it is	Key points
SOAP	Protocol for XML messages (envelope, header, body) usually over HTTP.	Strict contract, built-in standards (WS-Security, WS-ReliableMessaging, transactions); verbose and heavier.
WSDL	XML document describing a SOAP service: operations, messages, endpoints.	Machine-readable contract that allows client code to be generated.
UDDI	Registry for publishing and discovering web services.	Rarely used today; replaced by API catalogues and gateways.
REST	Architectural style using URLs as resources and HTTP methods (GET, POST, PUT, DELETE).	Lightweight, JSON or XML, stateless, cacheable, easy for mobile and web; security through HTTPS and OAuth2.

5.3 Three university functions exposed as web services

The three services below use the same endpoints as the proof-of-concept in Task 9. The sample responses were produced by running that code.

Service 1: Result lookup

Item	Definition
Endpoint	/api/v1/results/{reg_no}
Method	GET
Used by	Students, mobile app, partner colleges

Sample request and response

```
GET /api/v1/results/21CS001 HTTP/1.1
Host: api.university.example
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json

{"gpa":8.7,"name":"Asha K","reg_no":"21CS001","semester":5}

(not found) HTTP/1.1 404 NOT FOUND
{"error":"Student not found","reg_no":"99XX999"}
```

Service 2: Fee payment

Item	Definition
Endpoint	/api/v1/fees/pay
Method	POST
Used by	Mobile app, affiliated colleges, parents

Sample request and response

```
POST /api/v1/fees/pay HTTP/1.1
Content-Type: application/json
Authorization: Bearer <token>

{"reg_no":"21CS001","amount":45000}

HTTP/1.1 201 CREATED
{"amount":45000,"receipt_id":"RCPT-21CS001-0001","status":"SUCCESS"}
```

Service 3: Library search

Item	Definition
Endpoint	/api/v1/library/search?q={keyword}
Method	GET
Used by	Students, staff, partner libraries

Sample request and response

```
GET /api/v1/library/search?q=cloud HTTP/1.1

HTTP/1.1 200 OK
{"books":[{"available":true,"id":1,"title":"Cloud Computing: Concepts and
Design"},
{"available":true,"id":3,"title":"Cloud Native
Patterns"}],"count":2,"query":"cloud"}
```

Equivalent SOAP form (for comparison)

SOAP request for result lookup

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Body>
    <GetResult
xmlns="http://university.example/sis"><RegNo>21CS001</RegNo></GetResult>
  </soap:Body>
</soap:Envelope>
```

5.4 SOAP or REST for each function

Table 14: Choice of SOAP or REST per function

Function	Choice	Justification
Result lookup	REST	Simple read-only query; JSON is small and fast for mobile; GET responses can be cached by the CDN, which is essential for result-publication peaks.
Fee payment	REST (HTTPS, OAuth2, idempotency key)	Mobile app and 40 colleges need a lightweight, easy-to-integrate API. Security comes from TLS, tokens and a PCI-DSS compliant payment gateway (the university never stores card data). SOAP with WS-Security would only be used where a legacy bank interface requires it, placed behind the REST API.
Library search	REST	Simple read query, no transaction or formal contract needed; caching improves speed.

TASK 6 - Analyse the Service Models: IaaS, PaaS and SaaS (5 Marks)

6.1 Definitions and responsibility layers

- **IaaS (Infrastructure as a Service):** the provider supplies virtualized compute, storage and networking; the customer installs and manages the OS, middleware and applications.
- **PaaS (Platform as a Service):** the provider also manages OS, runtime and middleware; the customer deploys application code and data.
- **SaaS (Software as a Service):** the provider runs the complete application; the customer only uses it and manages users and configuration.

Table 15: Who manages which layer (C = customer/university, P = provider)

Layer	On-premise	IaaS	PaaS	SaaS
Application	C	C	C	P
Data	C	C	C	C (own data and settings)
Runtime	C	C	P	P
Middleware	C	C	P	P
Operating system	C	C	P	P
Virtualization	C	P	P	P
Servers	C	P	P	P
Storage	C	P	P	P
Networking	C	P	P	P

6.2 Categorisation of the five applications

Table 16: Mapping of university applications to service models

Application	Chosen model	Short reason
Student Information System (SIS)	IaaS first, then PaaS	Custom monolith; rehost on VMs, then move to managed app platform and DBaaS.
Learning Management System (LMS)	SaaS	Commodity software; vendor handles exam-week scaling.
Library System	SaaS	Small, standard, low customisation.
Exam Portal	PaaS	Custom, highly spiky; auto-scaling platform with managed database.
Email	SaaS	Standard service; no benefit from running it ourselves.

Reasoning behind each decision:

SIS - IaaS first, PaaS later. The SIS is a large, custom-built monolith that holds student records and depends on a specific operating system and database version. Moving it to

PaaS immediately would require code changes and re-testing. It is therefore rehosted on IaaS virtual machines (full control, minimum change) with the database moved to a managed database service. Once the code is modularised and exposed through APIs, it moves to PaaS to remove OS patching and gain auto-scaling. The final state is PaaS.

LMS - SaaS. A learning management system (for example Moodle or Canvas) is standard software with little need for custom infrastructure. A hosted SaaS LMS scales automatically during exam and assignment weeks, and the vendor provides availability and upgrades. Running our own LMS would repeat the problems of idle servers and peak failures. Customisation needs are handled through configuration and plug-ins.

Library - SaaS. The library system has small, predictable load and standard functions (catalogue, issue and return, search). A hosted library service removes the need for a dedicated server and administrator at low cost. Its search function is also exposed through the vendor's API.

Exam Portal - PaaS. This is the most elastic workload: nearly no use for most of the year and heavy load during exams. A PaaS (managed web platform with auto-scaling and a managed database) lets developers keep custom exam logic while the provider handles servers, patches and scaling. IaaS would need us to build and tune scaling ourselves; SaaS would not provide the required custom features. Where proctoring components need special software, they can run as containers on the same platform.

Email - SaaS. Email is a commodity. Services such as Google Workspace for Education or Microsoft 365 give high availability, spam filtering, security and storage at low cost. Running a private mail server would cost more and be less reliable.

6.3 Comparison of the models

Table 17: IaaS, PaaS and SaaS compared

Criterion	IaaS	PaaS	SaaS
Control	Highest (OS and above)	Medium (code and data)	Lowest (configuration only)
Cost	Pay for VMs; extra staff cost for administration	Moderate; less staff effort	Predictable per-user subscription
Scalability	Possible but must be configured (auto-scaling groups)	Built-in auto-scaling	Handled fully by vendor
Maintenance effort	High (OS, patches, middleware)	Low to medium (application only)	Very low
Vendor lock-in	Lowest (VMs are portable)	Medium (platform-specific services)	Highest (data and features tied to vendor)

6.4 Real provider examples

Table 18: One real provider example per model

Model	Example
IaaS	Amazon EC2 (AWS), Microsoft Azure Virtual Machines, Google Compute Engine

Model	Example
PaaS	Azure App Service, Google App Engine / Cloud Run, AWS Elastic Beanstalk
SaaS	Google Workspace, Microsoft 365, Salesforce

TASK 7 - Examine XaaS (3 Marks)

7.1 What XaaS is and four or more variants

XaaS (Anything as a Service) is the general idea that any IT capability - not only infrastructure, platform and software - can be delivered over the internet as a metered, on-demand service. Providers manage the service and the customer pays for use. It extends IaaS, PaaS and SaaS to specialised areas.

Table 19: XaaS variants and their relevance to the university

Variant	Description	Relevance
DBaaS (Database as a Service)	Managed database with automated backups, patching, replication and scaling.	Stores SIS, fee and exam data with high availability.
DRaaS (Disaster Recovery as a Service)	Replication of data and systems to a provider's site for fast failover.	Provides the missing disaster-recovery site.
FaaS (Function as a Service, serverless)	Code runs on events and is billed per execution.	Sending receipts, SMS/e-mail alerts, report generation.
BaaS / MBaaS (Backend as a Service)	Ready-made backend (authentication, push notifications, storage) for apps.	Speeds up the mobile app.
DaaS (Desktop as a Service)	Virtual desktops delivered from the cloud.	Lab computers and remote staff access.
Security as a Service	Cloud-delivered firewall, WAF, DDoS protection, identity and monitoring.	Protects public APIs and portals.

7.2 Two selected XaaS offerings and justification

1. DBaaS. Student records, fees and exam data are the university's most valuable assets. A managed database provides automatic backups, point-in-time recovery, multi-zone failover and read replicas. This avoids database administration effort, supports the 99.9% availability target and scales with load. Open engines (PostgreSQL or MySQL) are chosen to reduce lock-in.

2. DRaaS. The university currently has no disaster-recovery site, so a single failure could stop admissions or exams. Building a second data centre is too costly, but DRaaS replicates data to another region and charges mainly for storage until a failover is needed. It gives a clear recovery objective (for example RPO 15 minutes, RTO 4 hours in the original design) at a small fraction of the cost of an own DR site.

Other services (CDN/WAF security and functions for notifications) are supporting choices, but DBaaS and DRaaS give the greatest benefit for cost and risk.

7.3 Risks of over-reliance on many 'as-a-Service' providers

- **Vendor lock-in:** proprietary APIs make switching costly.
- **Cascading failures and availability:** the overall availability is the product of all providers' availability, and an outage of one service can stop the whole chain.
- **Integration complexity:** many providers mean many contracts, interfaces, identity systems and billing models.
- **Security and compliance:** data spreads across providers, increasing the attack surface and making privacy audits harder.
- **Cost creep and loss of skills:** small subscriptions add up, and in-house skills may fade.

Mitigation: use open standards, keep an exit plan and regular exports of data, limit the number of providers, use a single identity system (SSO), monitor costs and review SLAs.

TASK 8 - Design the Cloud Solution Architecture (5 Marks)

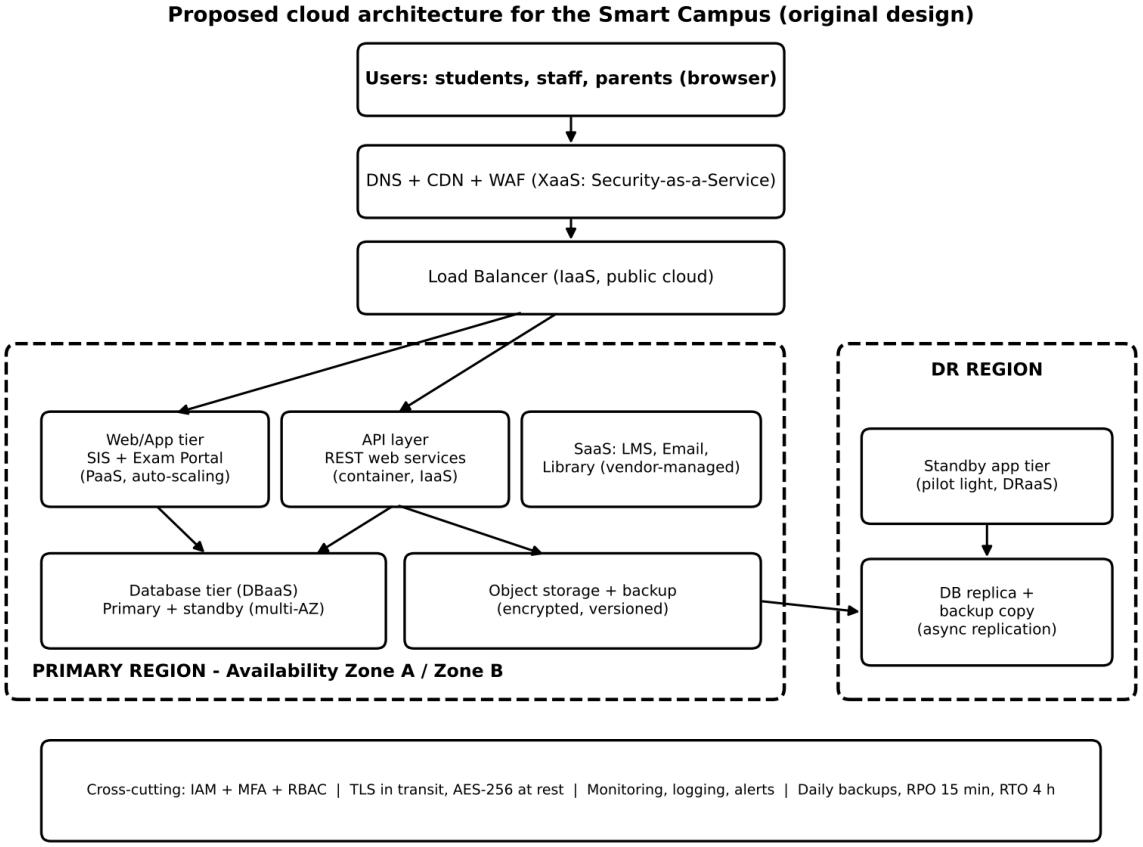


Figure 2: Proposed cloud architecture for the Smart Campus (original design)

8.1 Component justification and chosen models

Table 20: Deployment model, service model and justification for each component

Component	Deployment / service model	Justification
Users and DNS/CDN/WAF	Public cloud; Security-as-a-Service	Delivers static content near users, absorbs load and filters attacks before they reach the applications.
Load balancer	Public cloud; IaaS	Spreads traffic across servers in several zones and removes failed servers from rotation.
Web/application tier (SIS, Exam Portal)	Public cloud; PaaS with auto-scaling	Elastic capacity for peaks; provider manages OS and runtime.
API layer (REST web services)	Public cloud; containers on IaaS/PaaS	Exposes result lookup, fee payment and library search to mobile apps and colleges; containers scale fast.
LMS, Email, Library	Public cloud; SaaS	Commodity applications managed by vendors.
Database tier	Public cloud; DBaaS, multi-zone	Automatic backup, failover and patching for critical data.
Storage and backup	Public cloud; object storage	Cheap, durable and versioned storage for documents, logs and backups.
DR region	Public cloud; DRaaS (pilot light)	Second region holds replicated data and a minimal standby tier, giving disaster recovery without full cost.
Campus link	Hybrid (VPN)	Keeps remaining on-premise systems connected during migration.

8.2 Elasticity for admissions and exam peaks

- **Scheduled scaling:** the academic calendar is known, so capacity is increased a day before admissions, result publication and exams (for example minimum 6 instances instead of 2).
- **Reactive scaling:** if average CPU stays above 60% for 3 minutes (or request latency rises), add instances; remove them when CPU stays below 30% for 15 minutes. Range: minimum 2, maximum 20 instances, spread across zones.
- **Caching and CDN:** results pages and static files are cached, which reduces server load during result publication.
- **Queueing:** non-urgent tasks (receipts, notifications) are placed in a queue so that spikes do not slow core services.
- **Cost control:** budget alerts and a maximum instance limit prevent runaway bills.

8.3 Security and availability measures

Table 21: Security and availability measures

Area	Measures
Access control	Single sign-on, multi-factor authentication for staff, role-based access control, least privilege, separate admin accounts.
Encryption	TLS for data in transit; AES-256 encryption at rest with managed keys.
Network security	Private subnets for databases, security groups, WAF and DDoS protection.
Backup	Daily full and 15-minute incremental/transaction-log backups, versioned copies in a second region, restore tests every quarter.
Multi-zone deployment	Application and database run in at least two availability zones; failure of one zone does not stop service.
Disaster recovery	Warm/pilot-light DR region; RPO 15 minutes, RTO 4 hours; yearly DR drill.
Monitoring	Central logging, metrics, alerts and audit trails.

TASK 9 - Hands-on Implementation and Documentation (4 Marks)

This proof-of-concept hosts a small REST API for the university (result lookup, fee payment, library search) in a container, and deploys the same container to a managed PaaS service. The code in this section was executed and the outputs shown in Section 9.4 were captured from a real run of that code on a local machine. **The Docker build, the cloud deployment and the screenshots must be done on your own machine and account** and pasted into the screenshot boxes in Section 9.6, because they cannot be created on your behalf.

9.1 Environment

Table 22: Environment details

Item	Details
Language / framework	Python 3 with Flask (REST API), Gunicorn as production server
Container tool	Docker (Docker Desktop on Windows/macOS or Docker Engine on Linux)
Cloud (PaaS)	Google Cloud Run free tier (alternatives: Azure App Service free tier, Render)
Test clients	curl and a browser (Postman can also be used)

9.2 Source code

app.py

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory sample data (stands in for the cloud database in this proof of
concept)
RESULTS = {
```

```
    "21CS001": {"name": "Asha K", "semester": 5, "gpa": 8.7},
    "21CS002": {"name": "Ravi M", "semester": 5, "gpa": 7.9},
}
BOOKS = [
    {"id": 1, "title": "Cloud Computing: Concepts and Design", "available":
True},
    {"id": 2, "title": "Distributed Systems Principles", "available": False},
    {"id": 3, "title": "Cloud Native Patterns", "available": True},
]


@app.get("/health")
def health():
    return jsonify(status="UP"), 200


@app.get("/api/v1/results/<reg_no>")
def get_result(reg_no):
    record = RESULTS.get(reg_no)
    if record is None:
        return jsonify(error="Student not found", reg_no=reg_no), 404
    return jsonify(reg_no=reg_no, **record), 200


@app.get("/api/v1/library/search")
def search_library():
    q = request.args.get("q", "").lower()
    hits = [b for b in BOOKS if q in b["title"].lower()]
    return jsonify(query=q, count=len(hits), books=hits), 200


@app.post("/api/v1/fees/pay")
def pay_fee():
    body = request.get_json(silent=True) or {}
    if "reg_no" not in body or "amount" not in body:
        return jsonify(error="reg_no and amount are required"), 400
    return jsonify(status="SUCCESS", receipt_id="RCPT-" + body["reg_no"] + "-
0001",
                   amount=body["amount"]), 201


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
```

requirements.txt

```
flask==3.1.*
gunicorn==22.*
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 8080
CMD ["gunicorn", "-b", "0.0.0.0:8080", "-w", "2", "app:app"]
```

9.3 Configuration and deployment steps

Part A - Create and run the container (virtualization, OS-level)

```
mkdir uni-api && cd uni-api          # place app.py, requirements.txt, Dockerfile
here
docker build -t uni-api:1.0 .
```

```
docker run -d -p 8080:8080 --name uni-api uni-api:1.0
docker ps                          # screenshot: container is running
docker logs uni-api                # screenshot: gunicorn started
```

Part B - Deploy to a PaaS (Google Cloud Run)

- Create a free Google Cloud account and a project; enable the Cloud Run and Cloud Build APIs (a billing account is required, but usage for this test stays in the free tier).
- Install and log in to the gcloud CLI, then run the commands below from the project folder.

```
gcloud auth login
gcloud config set project <YOUR_PROJECT_ID>
gcloud run deploy uni-api --source . --region asia-south1 --allow-unauthenticated
# the command prints a Service URL, for example https://uni-api-xxxxx-
el.a.run.app
```

- Test the Service URL with the commands in Section 9.4 by replacing http://localhost:8080 with the Service URL.
- Delete the service after the assignment (gcloud run services delete uni-api) to avoid charges.

Optional SaaS/DBaaS component: a free-tier managed database (for example Cloud Firestore, MongoDB Atlas free tier or a Supabase PostgreSQL project) can replace the in-memory data. Record the connection steps and a screenshot of the console if you add it.

9.4 Tests and recorded output

The following output was captured from a real run of the code above (Flask server on port 8080, tested with curl). The Date and Server headers were removed for brevity.

Table 23: Test cases and results

No.	Test	Expected	Actual result
1	GET /health	HTTP 200, status UP	Passed - HTTP 200, {"status":"UP"}
2	GET /api/v1/results/21CS001	HTTP 200 with student data	Passed - HTTP 200, JSON with name, semester, gpa
3	GET /api/v1/results/99XX999 (failure case)	HTTP 404 error message	Passed - HTTP 404, {"error":"Student not found"}
4	GET /api/v1/library/search?q=cloud	2 matching books	Passed - count = 2
5	POST /api/v1/fees/pay	HTTP 201 with receipt id	Passed - HTTP 201, receipt RCPT-21CS001-0001

Terminal output (curl)

```
$ curl -i http://localhost:8080/health
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 16
Connection: close

{"status":"UP"}
```

```
$ curl -i http://localhost:8080/api/v1/results/21CS001
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 60
Connection: close

{"gpa":8.7,"name":"Asha K","reg_no":"21CS001","semester":5}

$ curl -i http://localhost:8080/api/v1/results/99XX999   (failure case)
HTTP/1.1 404 NOT FOUND
Content-Type: application/json
Content-Length: 49
Connection: close

{"error":"Student not found","reg_no":"99XX999"}

$ curl "http://localhost:8080/api/v1/library/search?q=cloud"
{"books":[{"available":true,"id":1,"title":"Cloud Computing: Concepts and
Design"},{"available":true,"id":3,"title":"Cloud Native
Patterns"}],"count":2,"query":"cloud"}

$ curl -X POST -H "Content-Type: application/json" -d
{reg_no:21CS001,amount:45000} /api/v1/fees/pay
HTTP/1.1 201 CREATED
Content-Type: application/json
Content-Length: 69
Connection: close

{"amount":45000,"receipt_id":"RCPT-21CS001-0001","status":"SUCCESS"}
```

9.5 Issues you may face and their fixes

These are common problems with this kind of set-up; replace this list with the issues you actually faced.

- **Port already in use:** another program uses 8080. Change the host port (for example -p 8081:8080).
- **Container exits immediately:** check docker logs; usually a missing file in the COPY step or a typo in requirements.txt.
- **Cloud Run deploy fails:** APIs not enabled or billing not linked; enable Cloud Run and Cloud Build and retry.
- **Service not reachable (403):** the service was not deployed with --allow-unauthenticated.
- **Container listens on the wrong port:** Cloud Run sends traffic to the port in the PORT variable (8080 by default); keep the container on 8080.

9.6 Screenshots to attach (annotate each one)

Screenshot 1: 'docker build' and 'docker ps' showing the running container (insert here)

Screenshot 2: browser/curl test of /api/v1/results/21CS001 on localhost (insert here)

Screenshot 3: Cloud Run (or other PaaS) console showing the deployed service and its URL (insert here)

Screenshot 4: test of the public Service URL, including the 404 failure case (insert here)

TASK 10 - Evaluate and Recommend the Final Solution (7 Marks)

10.1 Comparison of the three options

Option A: refresh and continue **on-premise**. Option B: **IaaS-based** (lift and shift of all servers to cloud VMs). Option C: **PaaS/SaaS-based** (SaaS for email, LMS and library; PaaS and DBaaS for SIS and Exam Portal).

Table 24: Comparison of on-premise, IaaS-based and PaaS/SaaS-based options

Criterion	A: On-premise	B: IaaS-based	C: PaaS/SaaS-based
Cost model	High CapEx (hardware), fixed OpEx	Mostly OpEx, small CapEx for migration	Mostly OpEx, moderate one-time refactoring
Scalability	Poor; 6-8 weeks to add servers	Good, but scaling must be configured	Best; built-in auto-scaling
Availability	Single data centre, no DR	Multi-zone possible, DR to be built	Multi-zone and DR built in
Security	Fully in own control, but limited skills and tools	Shared responsibility; we manage OS security	Provider manages platform; we manage app and data
Effort (admin)	Highest (3 staff in estimate)	High (2 staff, OS patching)	Lowest (about 1.25 staff)
Vendor lock-in	None	Low	Medium to high
Time to deliver	Slow	Fastest migration	Slower for custom apps

10.2 Three-year cost comparison

Assumptions (INR lakh): costs are in lakh; no price inflation or discounting; the same user numbers as today; staff cost is Rs 8 lakh per full-time person per year; hardware refresh cycle is longer than 3 years so all hardware is counted in year 0; option A includes no disaster-recovery site and no extra capacity for peaks (which would add cost); prices are estimates to be checked with provider calculators.

Table 25: Annual and three-year cost estimate (INR lakh)

Cost item	A: On-premise	B: IaaS	C: PaaS/SaaS
One-time CapEx: 12 new servers, 6 lakh each	72.0	-	-
One-time migration / refactoring / training	-	15.0	30.0
Compute: reserved VMs / platform	-	14.4	18.0 (PaaS)
Burst capacity for peak weeks	-	6.0	included in PaaS
SaaS subscriptions (email 3, LMS 12, library 3)	-	-	18.0
Storage, backup, CDN	-	5.0	3.0
Network and egress	6.0	4.0	-
Disaster recovery	-	6.0	4.0
Power, cooling, space	12.0	-	-
Hardware maintenance (AMC, 10%)	7.2	-	-
Software licences (OS, database)	10.0	10.0	-
Monitoring and security tools	-	3.0	3.0
Staff (A: 3, B: 2, C: 1.25 persons)	24.0	16.0	10.0
Training	-	-	2.0
Annual running cost (OpEx)	59.2	64.4	58.0
3-year OpEx	177.6	193.2	174.0
3-year total (CapEx + one-time + OpEx)	249.6	208.2	204.0
CapEx / one-time share of total	28.8%	7.2%	14.7%

Working: A = 72 + (3 x 59.2) = 72 + 177.6 = 249.6; B = 15 + (3 x 64.4) = 15 + 193.2 = 208.2; C = 30 + (3 x 58.0) = 30 + 174.0 = 204.0. Option C costs about Rs 45.6 lakh (18%) less than Option A and about Rs 4.2 lakh less than Option B over three years.

Cost alone does not separate B and C (the difference is within estimation error). The decision therefore rests on scalability, availability, DR and staff effort, where C is clearly better, and on risk, where C is weaker (lock-in).

10.3 Main risks and mitigation

Table 26: Risk register and mitigation steps

Risk	Impact	Mitigation
Vendor lock-in	High switching cost, price rises	Use containers, open database engines (PostgreSQL/MySQL) and standard APIs; keep regular data exports and an exit plan; avoid proprietary-only services for core data.
Data privacy and compliance	Leakage of student and fee data; legal penalties	Use an India region; encrypt at rest and in transit; role-based access and audit logs; contractual data-protection terms; no card data stored (use a PCI-DSS payment gateway).
Downtime	Missed admissions or exams	Multi-zone design, DR region, health checks, change freeze during exam weeks, tested rollback, SLAs with credits.
Skills gap	Misconfiguration, high cost	Provider training and certification, partner support during the first six months, infrastructure-as-code, budget alerts.
Migration failure / data loss	Service disruption	Phased migration, pilot first, backups before each step, parallel run and rollback plan.
Unexpected costs	Budget overrun	Budgets and alerts, tagging per application, reserved capacity for base load, regular cost reviews.

10.4 Final recommendation

Recommended solution: Option C, a phased PaaS/SaaS-based public-cloud solution with a temporary hybrid link, using IaaS only as a stepping stone for the SIS. The recommendation follows from the earlier tasks:

- Task 1 and Task 4: average utilisation is 15% and peaks need about 10 hosts against 3-4 needed normally, so owning capacity is wasteful; elastic public cloud fits.
- Task 6: email, LMS and library are commodity applications best bought as SaaS; the custom SIS and Exam Portal fit IaaS then PaaS.
- Task 7 and Task 8: DBaaS and DRaaS provide high availability and the missing DR site at low cost.
- Task 10 costs: lowest three-year cost (Rs 204 lakh versus Rs 249.6 lakh on-premise), the lowest staff effort and the lowest CapEx.

No single provider is chosen merely because it is popular. The selection criteria are: an India region, SLA of at least 99.9%, open database engines, container support, education pricing and exportable data. Any provider that meets them (AWS, Azure or Google Cloud) can be shortlisted, and a pilot with the PoC from Task 9 on two providers should decide the final choice.

10.5 Migration roadmap

Table 27: Phased migration roadmap

Phase	Timing	Activities	Reason for order
0. Preparation	Weeks 1-4	Cloud account, landing zone, networking and VPN, identity (SSO/MFA), budgets, training, inventory of the 12 servers.	Foundation for everything else.
1. Email	Months 1-2	Move mailboxes to SaaS email; keep old server read-only for 30 days.	First, because it is standard, has no code dependencies, is low-risk and gives a quick win and experience.
2. Library and LMS	Months 2-3	Migrate catalogue data and course content to SaaS; test with a pilot course.	Moderate risk, big benefit at exam time, completed before the next exam cycle.
3. SIS and Exam Portal (rehost)	Months 3-5	Rehost SIS on VMs, move databases to DBaaS, containerise Exam Portal, add API layer, load balancer, DR replication.	Most critical systems, so moved after the team has experience; no migration during admissions or exam weeks (change freeze).
4. Optimise and retire	Months 5-9	Move SIS to PaaS, tune auto-scaling, run DR drill, decommission old servers, review costs.	Removes the old hardware and realises the savings.

TASK 11 - Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 Marks)

Changed requirement: a mobile app and 40 affiliated colleges using student-record and fee-payment services, 10 times the current peak load, 99.9% availability and a limited operating budget.

11.1 Impact Analysis (2 Marks)

What has changed. The system was designed for one university with 27,000 internal users and a campus-focused web portal. It must now serve a mobile app, the public internet and 40 external organisations. Fee payment (money) and student records (personal data) become externally exposed, and the cost of downtime rises because many institutions depend on the service.

Table 28: Original assumptions affected by the new requirement

Assumption	Original	New	Effect
Load	About 200 requests/s at peak	About 2,000 requests/s at peak (10x)	Capacity, database and network must be re-sized.
Users	27,000 internal users	Plus mobile users and users of 40 colleges (for example about 80,000 more if each college has about 2,000 students - assumption)	Many more concurrent sessions and identities.

Assumption	Original	New	Effect
Availability	Best effort, DR RTO 4 h	99.9% (about 8.76 hours of downtime per year)	Needs multi-zone active-active design and faster recovery.
Cost	Moderate budget	Limited budget despite 10x load	Needs efficiency (caching, serverless, scheduled scaling) and possibly cost recovery from colleges.
Security	Internal users	External organisations and public internet	Stronger authentication, API protection and tenant isolation.

Why web services and APIs become critical. The mobile app and each college cannot log in to the SIS screens; they need stable, secure, versioned programmatic interfaces. APIs become the main entry point of the whole system, so they must be authenticated, rate-limited, monitored, cached and documented. Any failure or breaking change in an API now affects 40 organisations.

Multi-tenant security and isolation. Forty colleges share one platform, so a college must see only its own students and fee data. The risks are data leakage between tenants, one tenant's heavy usage slowing others (noisy neighbour), and misuse of credentials. Therefore tenant identity must be enforced at every layer (API, application, database), with per-tenant quotas, audit logs and encryption.

11.2 Evaluation of the Original Design (2 Marks)

Table 29: Evaluation of the original design under the new conditions

Question	Finding
Can the service-model mix and virtualization plan handle 10x load?	Partly. SaaS (email, LMS, library) scales automatically, and the PaaS tier can scale, but the original plan assumed a single SIS VM pair and a modest database. 10x load (2,000 req/s) would overload the single database primary and the VM-based SIS tier, and the PaaS tier's fixed size would need to be about ten times larger.
Are auto-scaling and a single region enough for 99.9%?	No. Auto-scaling handles load but not failures of a region or of dependencies. Using the assumed component availabilities in 11.4, the original single-chain design gives about 99.84% (about 14 hours downtime per year), above the 8.76-hour budget. DR RTO of 4 hours means a single regional incident could use half of the yearly budget.
Is cost still acceptable?	No. Scaling the original design linearly (PaaS sized for the 10x peak, always on) would cost about Rs 229 lakh per year (see 11.4), almost four times the Rs 58 lakh budget of the original design.
Would another service model or XaaS give better results?	Yes. An API gateway (managed), container orchestration, FaaS for bursty tasks (receipts, notifications), managed cache and CDN, and DRaaS in warm-standby mode would improve both cost and availability.

11.3 Adaptation or Redesign (2 Marks)

Decision: MODIFY. The foundations are still right: public cloud, SaaS for email, LMS and library, DBaaS and DRaaS, a security-first design and the phased roadmap. A full redesign is not necessary and would waste work and budget. What must change is the capacity model, the availability design and the multi-tenant API layer. Simply retaining the original design is not justified because it fails the availability target and cost limit (11.2).

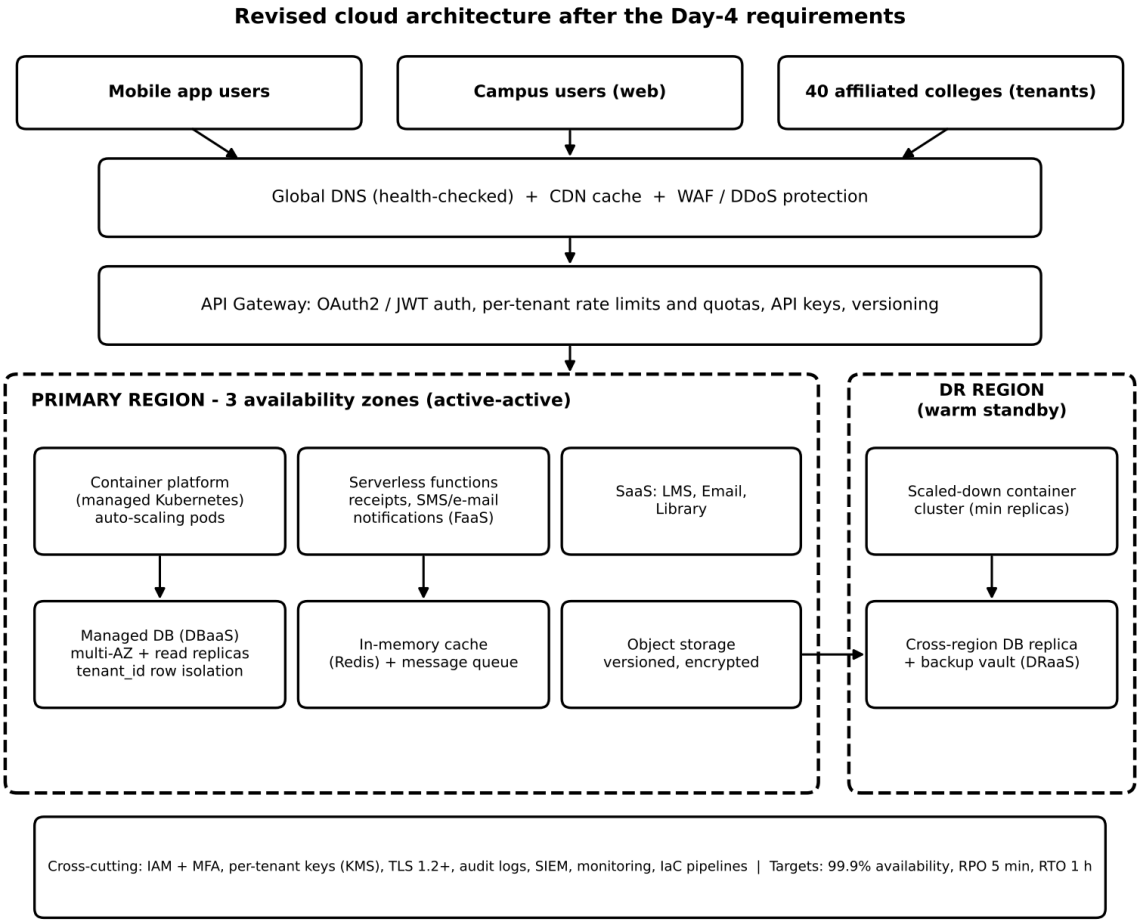


Figure 3: Revised cloud architecture after the Day-4 requirements

Table 30: What changes in the service-model mix and virtualization approach

Area	Original design	Revised design
Entry point	Load balancer	CDN + WAF + managed API Gateway (authentication, per-tenant rate limits, versioning)
Application tier	PaaS web/app tier and VMs for SIS	Containers on managed Kubernetes (or serverless containers) across 3 zones, auto-scaling on load; SIS modules (results, fees, records) split into services
Bursty tasks	In the application	FaaS (serverless) for receipts, SMS/e-mail notifications and report jobs; a message queue absorbs spikes
Database	DBaaS with standby	DBaaS multi-zone with read replicas and an in-memory cache (Redis); tenant_id isolation

Area	Original design	Revised design
Availability and DR	Two zones; pilot-light DR (RTO 4 h)	Three zones active-active; warm-standby DR region (RTO 1 h, RPO 5 min)
Virtualization	VMs and a few containers	Mostly containers (higher density, fast scaling); VMs only for the legacy SIS until refactored; managed services replace self-run VMs
Service-model mix	IaaS, PaaS, SaaS, DBaaS, DRaaS	Adds API-Gateway-as-a-Service, FaaS and managed cache; PaaS/CaaS replaces IaaS VMs; SaaS and DBaaS retained
Deployment model	Public with temporary hybrid link	Public cloud (hybrid link can be retired when campus systems are gone)

11.4 Evidence-Based Defence (2 Marks)

(a) Capacity calculation

Table 31: Capacity sizing for 10x load

Step	Working	Result
Peak request rate	200 req/s x 10	2,000 req/s
Capacity of one container instance (2 vCPU) at about 60% CPU (assumed from load test)	100 req/s per instance	-
Instances for peak load	2,000 / 100	20 instances
Survive loss of one of 3 zones: the remaining 2 zones must carry the full load	Each zone sized for 50% of peak = 1,000 req/s = 10 instances	10 per zone
Add 30% headroom	10 x 1.3	13 per zone, 39 total (maximum set to 40)
Normal day (about 10% of peak)	200 req/s / 100	About 2-3 instances per zone, minimum 6 in total
Database	About 2,000 req/s x 60% need the database x 3 queries = 3,600 queries/s; 15% writes = 540 writes/s	Primary handles writes; 2 read replicas plus cache handle reads

The sizing gives no credit for caching, so it is conservative. A CDN and cache hit ratio of even 50% on read-heavy calls (results, library search) would roughly halve the origin load.

(b) Availability calculation

99.9% availability allows 365 x 24 x 0.001 = **8.76 hours** of downtime per year. Component availabilities below are assumed, typical of published provider SLAs, and must be confirmed in the contract. For components in series, availabilities multiply; for redundant components in independent zones, unavailability multiplies.

Table 32: Availability of the original and revised designs

Design	Calculation	Availability	Downtime per year
Original: LB 99.99% x app tier 99.9% x DB 99.95% (single chain)	0.9999 x 0.999 x 0.9995	99.84%	About 14.0 hours (fails the target)
Revised: LB/gateway 99.99% x app tier (2 independent zones, each 99.9%: 1 - 0.001 x 0.001 = 99.9999%) x multi-zone DB 99.95%	0.9999 x 0.999999 x 0.9995	99.94%	About 5.3 hours (meets 99.9%)

The result is a planning estimate; in practice the database and shared dependencies limit the benefit of redundancy, which is why the DB is multi-zone, DR is warm, and failover is tested regularly.

(c) Cost comparison

Table 33: Annual running cost at 10x load (INR lakh)

Cost item	Original design scaled linearly	Revised design
SaaS (email, LMS, library)	18.0	18.0
Application compute	180.0 (PaaS sized 10x, always on)	26.0 (reserved base 12 + auto-scaling burst 14)
API gateway and WAF	-	5.0
Database	included above	12.0 (multi-zone + replicas)
Cache and queue	-	3.0
CDN and storage	12.0	7.0
Serverless functions	-	2.0
Disaster recovery	4.0 (pilot light)	8.0 (warm standby)
Monitoring and security	3.0	5.0
Staff	10.0	14.0 (adds SRE/DevOps skill)
Training	2.0	3.0
Annual total	229.0	103.0
3-year total (revised includes Rs 12 lakh one-time upgrade)	687.0	321.0 (103 x 3 + 12)

The revised design is about **Rs 126 lakh per year (55%) cheaper** than scaling the original design linearly, because capacity follows demand, caching reduces origin load, and serverless covers bursty tasks. It is Rs 45 lakh per year more than the original Rs 58 lakh plan, which is the price of 10x load, 99.9% availability and multi-tenancy. To keep the budget limited, the university can recover cost from the colleges: for example a subscription of Rs 1.5 lakh per college per year would give 40 x 1.5 = Rs 60 lakh per year, covering more than the extra Rs 45 lakh. All figures are estimates to be re-checked with provider calculators.

(d) Security and multi-tenancy

Table 34: Multi-tenant security and isolation controls

Layer	Control
Identity	OAuth2/OpenID Connect; each college has its own client credentials and users; tokens carry a tenant_id claim; MFA for administrators.
API gateway	Per-tenant API keys, rate limits and quotas (prevents noisy neighbours), schema validation, WAF rules, IP allow-lists where possible.
Application	Tenant context is taken from the token, never from the request body; authorisation checks on every request.
Database	tenant_id column on every table with row-level security policies (shared database, logical isolation); highly sensitive tenants can be moved to a separate schema or database if required.
Encryption	TLS 1.2+ everywhere; encryption at rest with per-tenant keys in a key management service.
Payments	Card data handled only by a PCI-DSS compliant payment gateway; the university stores only transaction references.
Monitoring and audit	Per-tenant audit logs and anomaly alerts; periodic penetration tests and access reviews; India-region storage for data-protection compliance.

(e) Trade-offs

Table 35: Trade-offs of the revised design

Trade-off	Issue	How it is managed
Vendor lock-in	Managed gateway, queue and cache services tie the design to one provider.	Use containers and Kubernetes, open-source engines (PostgreSQL, Redis) and infrastructure-as-code so the platform can be moved with reasonable effort.
Complexity	More components (gateway, orchestration, queues, multi-region) mean more skills and more failure modes.	Use managed services, automation, dashboards and runbooks; add one or two SRE/DevOps staff (already included in the cost).
Migration effort	Splitting the SIS into services and adding the gateway takes time (assumed Rs 12 lakh and about 3 months).	Strangler approach: put the API gateway in front of the existing SIS and move one function at a time (results first, then fees); change freeze during peaks.
Cost versus availability	Warm DR and three zones cost more than pilot light and two zones.	Scheduled scaling, reserved capacity for the base load, and the college subscription model offset the cost.

Final decision: MODIFY the original design as described in Section 11.3. It is supported by the capacity sizing (about 39 instances across three zones), the availability estimate (about 99.94%, inside the 99.9% target), the cost comparison (Rs 103 lakh per year versus Rs 229 lakh for linear scaling) and the security controls for 40 tenants. Before go-live, the capacity and availability assumptions should be verified with a load test and a failover test.

References

- Mell, P. and Grance, T., The NIST Definition of Cloud Computing, NIST Special Publication 800-145, 2011.
- Course material for CSA15 - Cloud Computing and Big Data Analytics (CO1); provider documentation for AWS, Microsoft Azure and Google Cloud.